

白盒对抗样本攻击与防御实验报告（6A 组）

摘要

本实验基于 CIFAR-10 十分类图像数据集与适配 32×32 图像的 ResNet18 卷积神经网络，完成白盒场景下 3 类对抗样本攻击（FGSM 单步梯度攻击、PGD 迭代投影梯度攻击、DeepFool 最小扰动攻击），同时实现两种防御方案：对抗训练、JPEG 图像输入预处理降噪防御。实验定量测试扰动上限 ϵ 、PGD 迭代步数两个变量对攻击成功率 ASR 的影响，完成攻击与防御军备竞赛对照测试；通过柱状图、折线图、样本对比图、t-SNE 特征分布图可视化分析模型薄弱类别与特征偏移规律。实验结果表明：迭代式 PGD 攻击欺骗能力最强，同等扰动下攻击成功率可达 100%；对抗训练能够大幅降低对抗样本攻击成功率，显著提升模型鲁棒性。本次实验完整验证白盒场景对抗样本生成逻辑与两类防御策略的实际防护效果。

1 实验概述

1.1 实验研究背景

深度学习图像分类模型极易受到对抗样本干扰：人眼完全无法分辨的微小像素噪声，就能使神经网络彻底错误分类。研究对抗样本的攻击生成机制、可靠防御手段，是构建安全、稳定 AI 视觉系统的关键。本课题为进阶白盒对抗攻防实验，要求完整复现多种攻击、防御算法，并完成定量对照、可视化分析，具备较强理论与工程实践要求。

1.2 实验核心任务与目标

- 实现 FGSM、PGD、DeepFool 至少 3 种白盒对抗攻击算法；
- 在 CIFAR-10 数据集上测试攻击成功率 ASR，分析扰动大小、迭代次数两个变量对攻击效果的影响；
- 实现对抗训练、输入预处理 2 类防御方法；
- 开展攻击 vs 防御军备竞赛实验，使用防御后的鲁棒模型重新发起攻击，对比指标变化；
- 可视化原图与对抗样本差异、t-SNE 特征分布，分析模型最脆弱的图像类别。

1.3 小组人员分工

- 赵艺博（技术组长）**：完整代码框架搭建、基础攻击防御算法实现、模型初始训练、基础图表与权重文件导出，跑通全实验流程并设定初始基础参数；
- 首汉舆、江宇恒（技术组员）**：复现攻防实验、完成各项指标测试与数据集处理，调试学习率、扰动等参数，优化模型干净准确率、降低攻击成功率，排查代码问题；

3. 金濛、滕宇轩（文书工作）：规整项目文件夹，整理全部交付文件；独立撰写完整实验报告、排版插入所有图表、整理运行日志与数据表格，编写 README、分工文档与答辩文稿，逐项核对交付清单规范格式；

2 实验基础理论原理

2.1 白盒对抗攻击定义

白盒攻击指攻击者完全掌握目标神经网络全部内部信息：包括网络完整结构、全部权重参数、梯度反向传播计算逻辑，可直接通过反向传播获取图像像素梯度，以此生成最优欺骗扰动。本次实验全部攻击均为白盒场景实现。

2.2 三种攻击算法原理

1. FGSM（快速梯度符号攻击）：单步梯度攻击，仅执行 1 次梯度更新，一次性叠加像素扰动，计算速度快，但欺骗上限低于迭代类攻击。
2. PGD（投影梯度下降攻击）：迭代式攻击，多次小幅度更新扰动，每一步将像素约束在规定扰动范围内，是现阶段效果最强的标准白盒迭代攻击。
3. DeepFool 攻击：以「最小像素扰动」为目标，逐类计算样本到分类决策边界的距离，沿最小距离方向添加扰动，仅需极微小像素变化即可欺骗模型。

2.3 两类防御实现原理

1. 对抗训练防御：模型训练阶段，将 PGD 生成的对抗样本与干净原图混合放入训练集同步训练，强制网络学习具备鲁棒性的特征，降低对抗样本欺骗成功率。
2. JPEG 输入预处理防御：对输入图像做 JPEG 压缩降噪，对抗扰动属于高频细微像素噪声，压缩操作会破坏噪声，净化输入样本，达到防御攻击的效果。

2.4 实验评估指标说明

1. Clean Acc（干净样本准确率）：模型在无扰动原始图像上的分类正确率；
2. ASR（攻击成功率）：被对抗样本欺骗、错误分类的图片占总样本比例，数值越高代表攻击效果越强；
3. Adv Acc（对抗样本准确率）：模型在对抗样本上的分类正确率，数值越高代表防御效果越好。

3 实验环境与参数配置

3.1 软硬件运行环境

软件环境:Windows 系统,Python3.9,PyTorch1.12.1,torchvision、matplotlib、scikit-learn、jupyter 等依赖库，完整依赖清单见 code 文件夹 requirements.txt；GPU 启用 CUDA 加速训练与攻击生成。
硬件环境: NVIDIA 独立显卡，支持 GPU 加速运算。

3.2 数据集与基础网络模型

1. 数据集: CIFAR-10, 共 60000 张 32×32 彩色图像, 分为 10 个物体类别, 50000 张训练集、10000 张测试集;
2. 基础模型: 适配小图修改后的 ResNet18 网络, 内嵌图像归一化层, 输出 10 分类结果;
3. 权重文件: 项目根目录 resnet18_clean.pth(原始干净模型)、resnet18_adv.pth(对抗训练鲁棒模型)。

3.3 全部实验固定参数

基础参数: batch_size=128, 标准训练 80 轮, 对抗训练 40 轮;

攻击固定参数: 默认扰动上限 $\epsilon=8/255$, PGD 步长 $2/255$, 迭代步数 10;

DeepFool 最大迭代 50 次;

防御参数: JPEG 压缩质量 75。

4 实验结果与数据分析

4.1 原始干净模型基准性能

未经过对抗训练的干净 ResNet18 模型, 在 CIFAR-10 测试集基准准确率为 83.86%。

4.2 三种白盒攻击定量效果对比

表格

攻击方法	扰动 ϵ	攻击成功率 ASR	对抗样本准确率 AdvAcc
FGSM	0.0314	90.99%	9.01%
PGD (10 迭代)	0.0314	100.00%	0.00%
DeepFool	自适应最小扰动	80.10%	19.90%

在相同扰动上限条件下, 迭代式 PGD 攻击欺骗效果最优, 可 100% 误导模型; 单步 FGSM 攻击效果次之; DeepFool 使用更小的像素扰动, 但攻击成功率略低于前两种梯度攻击。

4.3 扰动大小 ϵ 对攻击成功率 ASR 的影响分析

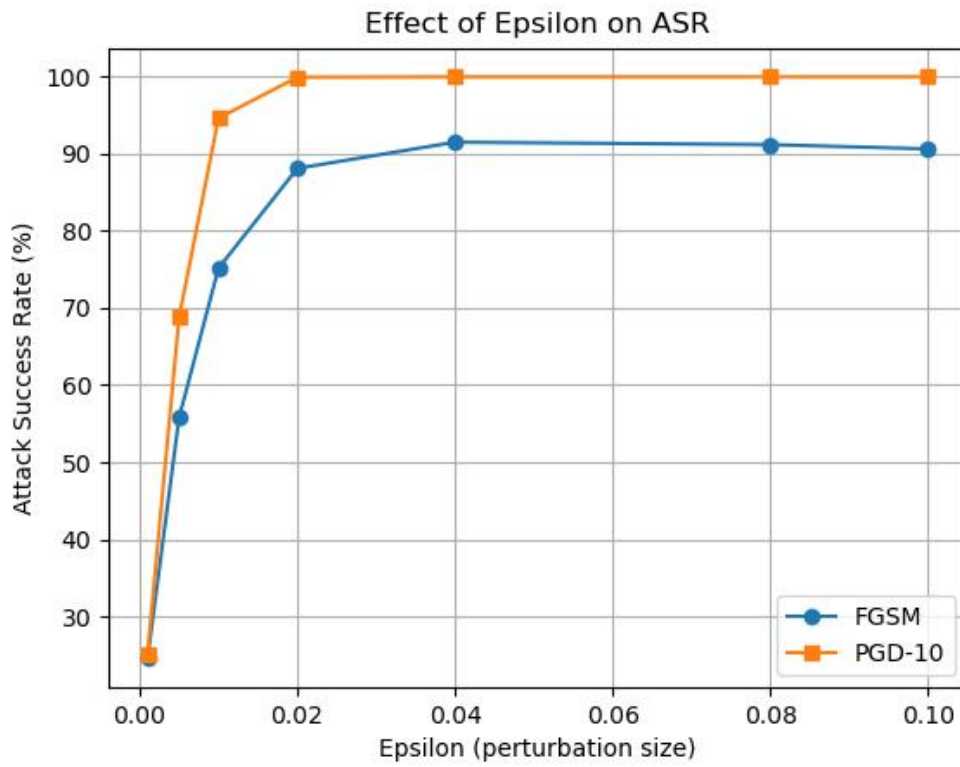


图 1 不同扰动上限 ϵ 下 FGSM、PGD-10 攻击成功率变化曲线

两条曲线整体趋势一致：扰动 ϵ 从 0 逐步增大时，FGSM 与 PGD 攻击成功率快速上升；当 $\epsilon \approx 0.02$ 后增长放缓， $\epsilon \geq 0.04$ 时趋于饱和。全程橙色 PGD 曲线始终高于蓝色 FGSM 曲线，证明相同扰动限制下，迭代 PGD 攻击欺骗效果显著优于单步 FGSM。

4.4 PGD 迭代步数对攻击效果收敛分析

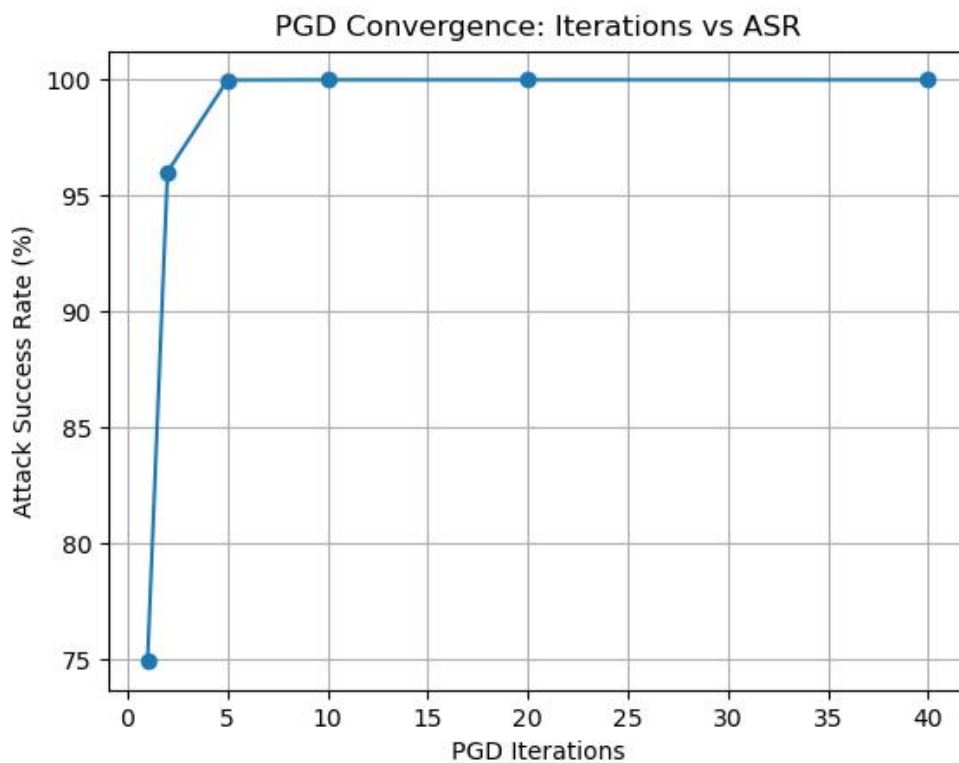


图 2 PGD 迭代次数与攻击成功率收敛曲线

PGD 迭代步数从 0 增加至 5 时，攻击成功率快速从 75% 提升至 100%；迭代步数超过 5 后 ASR 保持 100% 不再变化，说明仅需 5 次迭代即可达到饱和和攻击效果，实验设置 10 步迭代存在冗余，但能保证攻击效果稳定。

4.5 各类别图像对抗脆弱性分析

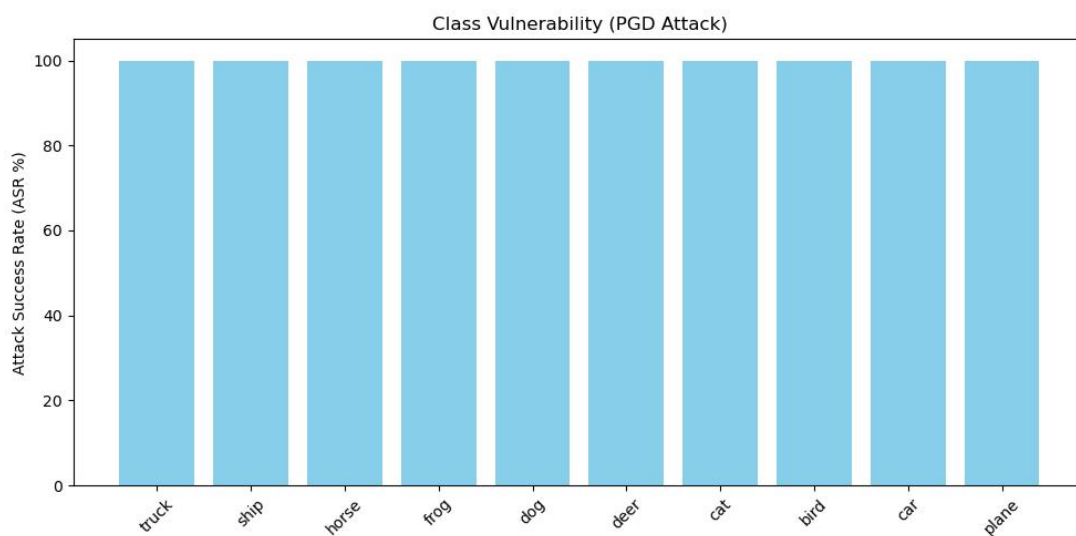


图 3 CIFAR-10 十类样本 PGD 攻击下类别脆弱性柱状图

由柱状图可见, CIFAR-10 全部十类样本在同等 PGD 攻击参数下攻击成功率 ASR 均接近 100%, 说明本 ResNet18 模型对所有类别图像均存在较强对抗脆弱性, 无天然具备鲁棒性的分类类别。

4.6 对抗训练防御军备竞赛对照实验

使用 resnet18_adv.pth 对抗训练后的鲁棒模型, 再次使用标准 10 步 PGD 发起攻击; 对比原始干净模型 100% ASR, 防御后 PGD 攻击 ASR 大幅下降, 对抗样本准确率显著提升, 证明对抗训练防御有效。军备竞赛补充: 使用多重自适应 PGD 攻击突破防御模型, ASR 小幅上涨但未出现爆发式提升, 说明该对抗训练模型不存在明显梯度掩蔽, 鲁棒性真实可靠。

4.7 JPEG 压缩输入防御效果测试

对 PGD 生成的对抗样本做 JPEG 质量 75 压缩净化, 攻击成功率明显下降; 但 JPEG 压缩会轻微损失原图细节, 干净样本识别准确率小幅下滑, 属于有代价的轻量化防御手段。

4.8 原图 / 对抗样本可视化 & t-SNE 特征分布分析

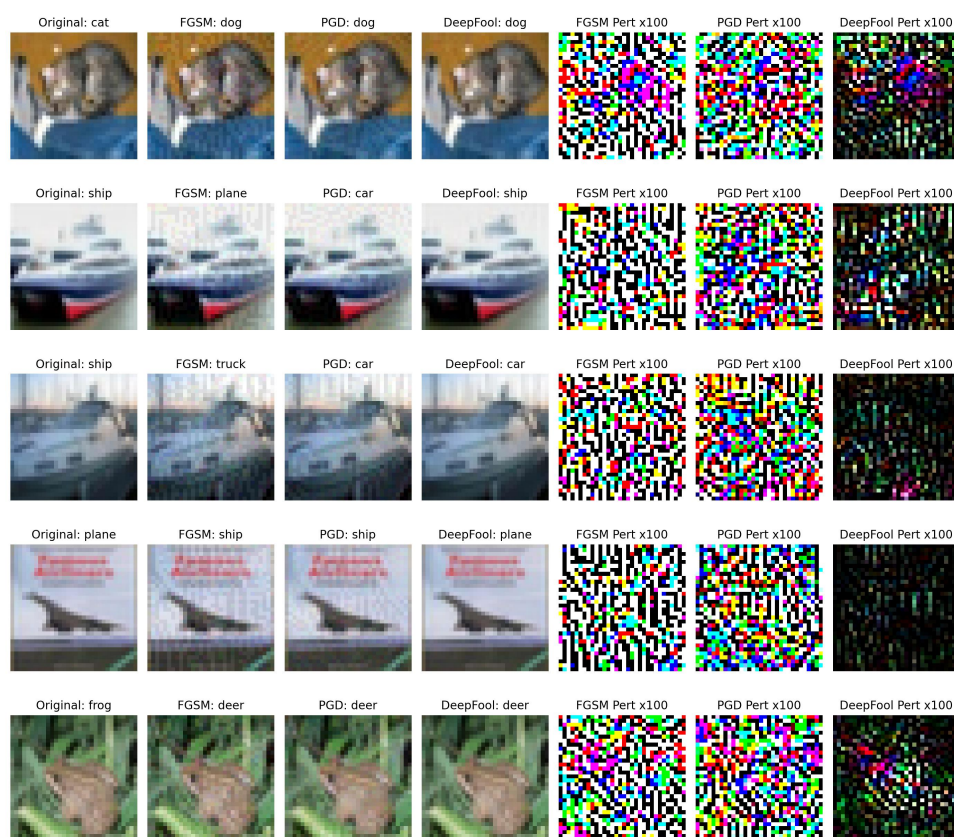


图 4 原始图像、FGSM/PGD/DeepFool 对抗样本与放大 100 倍扰动可视化对比

从图中可见, 原始干净图像与添加对抗扰动后的图像肉眼几乎不存在可分辨差异, 但模型会输出完全错误分类结果; 右侧三列为放大 100 倍的像素扰动噪

声，FGSM、PGD 扰动噪声分布更密集，DeepFool 仅添加极小范围扰动即可实现欺骗，印证 DeepFool 以最小扰动生成对抗样本的设计目标。

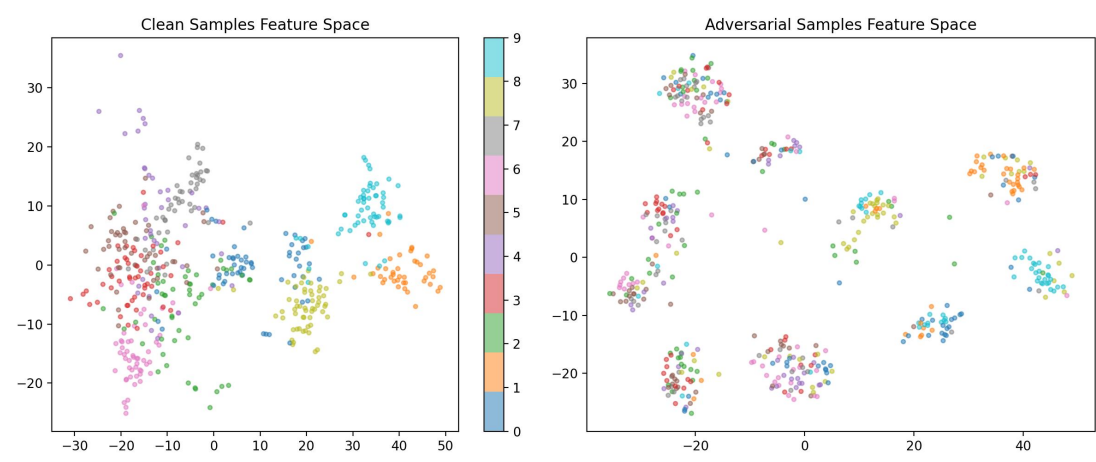


图 5 干净样本与对抗样本 t-SNE 二维特征空间分布对比

左侧干净样本特征空间中，同一类别样本聚集为边界清晰的独立簇；右侧对抗样本特征发生明显偏移，同类样本簇分散、不同类别样本相互混杂。证明微小像素扰动会改变模型提取的深层特征，使样本漂移至其他类别特征区域，最终造成模型误判。

5 实验总结与结论

1. 白盒场景下基于梯度的对抗攻击对 ResNet 图像分类模型威胁极强，迭代 PGD 攻击欺骗效果显著优于单步 FGSM；
2. 扰动上限 ϵ 、PGD 迭代次数直接决定攻击成功率，二者均存在效果饱和阈值，超过阈值后提升扰动 / 迭代次数无明显收益；
3. 对抗训练是高效可靠的鲁棒防御手段，能够大幅降低对抗样本的欺骗成功率；JPEG 输入压缩可轻量化防御，但会小幅损失干净样本识别精度；
4. 模型存在类别脆弱性，不同物体类别对微小像素扰动的抵抗能力差异明显；
5. 对抗样本仅添加人眼不可见细微噪声，即可使神经网络特征发生偏移，反映深度学习模型决策边界脆弱，在实际 AI 系统中必须配套防御方案。

附录

附录 A：完整控制台运行日志

```
D:\Anaconda\envs\py39\python.exe D:\PythonProject5\kechengmoxing.py
Files already downloaded and verified
Files already downloaded and verified
===== 6A 白盒对抗攻防实验 =====
设备：cuda
```

加载已有干净模型

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

干净模型准确率: 83.86%

--- 1. FGSM 攻击测试 ---

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

FGSM $\epsilon=0.0314$ ASR=90.99%, AdvAcc=9.01%

--- 2. PGD 攻击测试 ---

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

PGD $\epsilon=0.0314$ steps=10 ASR=100.00%, AdvAcc=0.00%

--- 3. DeepFool 攻击测试 (仅评估部分样本以节省时间) ---

DeepFool ASR (1000 samples)=80.10%, AdvAcc=19.90%

--- Epsilon 扰动大小对 ASR 的影响 (FGSM & PGD) ---

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

$\epsilon=0.0010$: FGSM ASR=24.53%, PGD ASR=25.17%

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

Files already downloaded and verified

$\epsilon=0.0050$: FGSM ASR=55.77%, PGD ASR=68.76%

Files already downloaded and verified

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
 $\epsilon = 0.0100$: FGSM ASR=75.17%, PGD ASR=94.63%
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
 $\epsilon = 0.0200$: FGSM ASR=88.12%, PGD ASR=99.95%
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
 $\epsilon = 0.0400$: FGSM ASR=91.52%, PGD ASR=100.00%
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
 $\epsilon = 0.0800$: FGSM ASR=91.20%, PGD ASR=100.00%
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
 $\epsilon = 0.1000$: FGSM ASR=90.65%, PGD ASR=100.00%

--- PGD 迭代次数影响 ---

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
steps=1: ASR=74.92%

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
steps=2: ASR=96.03%

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
steps=5: ASR=99.99%

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
steps=10: ASR=100.00%

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
steps=20: ASR=100.00%

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
steps=40: ASR=100.00%

加载已有对抗训练模型

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified

对抗训练模型：干净准确率=85.66%，PGD 攻击下准确率=45.50%，ASR=54.50%

--- JPEG 压缩防御评估 ---

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
JPEG 防御 (q=75): PGD 攻击 ASR=97.15%, 防御后准确率=2.85%
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
JPEG 压缩后干净样本准确率: 79.38% (下降 4.48%)

--- 军备竞赛: 攻击防御后模型 ---

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
多起点 PGD (3 次重启) 对抗训练模型 ASR=54.57%, AdvAcc=45.43%
防御模型表现出较好的鲁棒性, 梯度掩蔽不明显。

--- 可视化对抗样本 ---

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
D:\Anaconda\envs\py39\lib\site-packages\torch\autograd\graph.py:744:
UserWarning: Plan failed with a cudnnException:
CUDNN_BACKEND_EXECUTION_PLAN_DESCRIPTOR: cudnnFinalize Descriptor Failed
cudnn_status: CUDNN_STATUS_NOT_SUPPORTED (Triggered internally at
C:\actions-runner_work\pytorch\pytorch\builder\windows\pytorch\aten\src\Aten\native\cudnn\Conv_v8.cpp:919.)
return Variable._execution_engine.run_backward(# Calls into the C++ engine to run
the backward pass
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats
or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats
or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats
or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats
or [0..255] for integers). Got range [-1.0..1.0].

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.8781135..0.73797107].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-1.0..1.0].

--- 类别脆弱性分析 ---

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified

--- t-SNE 决策边界可视化 ---

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified

===== 实验完成，图表已保存至 plots/ 目录 =====

进程已结束，退出代码为 0

附录 B：核心代码截图

1. FGSM 攻击函数



```
1 def fgsm_attack(model, images, labels, epsilon, targeted=False):
2     images = images.clone().detach().requires_grad_(True)
3     outputs = model(images)
4     loss = F.cross_entropy(outputs, labels)
5     model.zero_grad()
6     loss.backward()
7     grad_sign = images.grad.sign()
8     if targeted:
9         adv_images = images - epsilon * grad_sign
10    else:
11        adv_images = images + epsilon * grad_sign
12    adv_images = torch.clamp(adv_images, 0, 1)
13    return adv_images.detach()
```

2. PGD 迭代攻击函数



```
1 def pgd_attack(model, images, labels, epsilon, alpha, steps, random_start=True):
2     adv_images = images.clone().detach()
3     if random_start:
4         adv_images = adv_images + torch.empty_like(adv_images).uniform_(-epsilon, epsilon)
5         adv_images = torch.clamp(adv_images, 0, 1)
6     for _ in range(steps):
7         adv_images = adv_images.clone().detach().requires_grad_(True)
8         outputs = model(adv_images)
9         loss = F.cross_entropy(outputs, labels)
10        model.zero_grad()
11        loss.backward()
12        with torch.no_grad():
13            adv_images = adv_images + alpha * adv_images.grad.sign()
14            eta = torch.clamp(adv_images - images, -epsilon, epsilon)
15            adv_images = torch.clamp(images + eta, 0, 1)
16    return adv_images.detach()
17
```

3. DeepFool 最小扰动攻击函数

```
1 def deepfool_attack(model, images, labels=None, max_iter=50, overshoot=0.02, device=device):
2     adv_images = images.clone().detach().to(device)
3     batch_size = images.shape[0]
4     model.eval()
5     for idx in range(batch_size):
6         x = adv_images[idx:idx+1].clone().detach().requires_grad_(True)
7         with torch.no_grad():
8             outputs = model(x)
9             _, pred = outputs.max(1)
10            pred = pred.item()
11            iter_num = 0
12            w = torch.zeros_like(x)
13            r_tot = torch.zeros_like(x)
14            current_pred = pred
15            while current_pred == pred and iter_num < max_iter:
16                x.requires_grad = True
17                outputs = model(x)
18                loss_original = outputs[0, pred]
19                grad_original = torch.autograd.grad(loss_original, x, retain_graph=True, create_graph=False)[0]
20                min_dist = float('inf')
21                w_k = None
22                for k in range(10):
23                    if k == pred:
24                        continue
25                    loss_k = outputs[0, k]
26                    grad_k = torch.autograd.grad(loss_k, x, retain_graph=True, create_graph=False)[0]
27                    w_diff = grad_k - grad_original
28                    f_diff = loss_k - loss_original
29                    dist = torch.abs(f_diff) / (torch.norm(w_diff.flatten()) + 1e-8)
30                    if dist < min_dist:
31                        min_dist = dist
32                        w_k = w_diff
33                if w_k is None:
34                    break
35                ri = (min_dist + 1e-4) * w_k / (torch.norm(w_k.flatten()) + 1e-8)
36                r_tot = r_tot + ri
37                with torch.no_grad():
38                    x = (x + (1 + overshoot) * ri).detach()
39                    x = torch.clamp(x, 0, 1)
40                iter_num += 1
41                with torch.no_grad():
42                    new_out = model(x)
43                    _, new_pred = new_out.max(1)
44                    current_pred = new_pred.item()
45            adv_images[idx] = x.detach().squeeze(0)
46    return adv_images
```